

프로그래밍 언어론

중간에서 다시 출제

```
void A(){
    int x,y,z;
    void B(){
        int y;
        x=z+1;
        y=0;
        z=y+2;
    }
    void C(){
        int z;
        void D(){
            int x;
            x=z+1;
            y=x+1;
            B();
        }
        z=5;
        D();
    }
    x=10;
    y=11;
    z=12;
    C();
    printf("%d, %d, %d, x, y, z")
}
```

- 1) Static Scope일 때의 출력을 구하라
- 2) Dynamic Scope일 때의 출력을 구하라

38-1번 문제 13, 7, 2

38-2번 문제 10, 7, 12

7장 - Expressions and the Assignment Statement 문장 내의 제어

문장을 평가하는 순위는 결합성(associativity)와 우선순위(precedence rules)에 의해 결정됨.
폰노이만 구조에서, 배정문(assignment)는 가장 기본적인 구문이다.

#산술 표현(Arithmetic expressions)

수학에서의 유사한 산술 평가식의 자동 평가가 최초의 고급 프로그램 언어의 주요 목표 중 하나였음.

#연산자 우선순위

- 서로 다른 precedence 수준의 연산자가 평가될 때 ; $A + B * C$
- 항등 연산자(identity operator) : 붙이나 붙이지 않나 영향X +, () 등 ; $+A, A + (-B) * C$
- 대부분의 언어에서 연산자 우선순위는 같음
- 언어들의 우선순위

FORTRAN : ******(지수) -> * , / -> +, -

Pasal : *, /, div, mod -> +,-

ANSI C : ++,--,unary +, -> *,/,% -> binary +,-

#결합 규칙(Associativity Rule)

- 같은 수준의 우선순위를 가진 연산자가 있으면, 언어의 결합 규칙에 따라서 작동함.

e.g. $A - B + C - D$

FORTRAN : 왼쪽부터 *, /, +, - 오른쪽부터 **

A-B-C (왼쪽부터) A**B**C(오른쪽부터)

Pascal : 전부 왼쪽부터

ANSI C : 왼쪽부터 *, /, %, binary +, - 오른쪽부터 : ++, --, unary +, -, =, +=(대입연산자)
a=b=c=10

APL : 연산자 우선순위가 없음. 오로지 결합규칙에 따름(오른쪽->왼쪽)

★수학에서의 법칙이 컴퓨터에서는 통용되지 않을 수 있음! 컴퓨터에서는 결합법칙이 성립하지 않을 수 있다

b.o. 오버플로우 등 문제(자료형의 표현 한계) 때문에

만약 컴파일러가 연산자들의 평가를 재정렬할 수 있다면,

e.g. $A + B + C + D$

A=INT_MAX B=10 C=-30 D=10

#괄호(Parentheses)

괄호를 사용함으로써 우선순위를 무시하고 연산 순서를 바꿀 수 있다.

e.g. $(A+B) * C$

#피연산자 평가 순서

side effect : 함수나 procedure 내에서 전역변수나 매개변수를 변경시키는 것

연산자의 피연산자에 **side effect**가 있으면, 피연산자 평가 순서를 정하는 것이 어려워진다.

해결책

함수의 side effect를 허용하지 않음 -> but, 유연성이 떨어짐

언어에서 처리 순서를 정의함 -> C와 Java에서는 이렇게 순서를 정의함.

```

procedure sub1(...) ;
  var a : integer;
  function fun(x: integer) : integer ;
    ....
    a := 27 ;
    return(5);
  end;
procedure sub2(...) ;
  ....
  a := 10 ;
  b := a + fun(b) ;
  print(b) ;
end;
end;

```

출력이 10+5가 되는가? 27+5가 되는가?

대부분의 언어는 10+5 임. 식에 들어온 순간 값이 결정

#조건 연산자

if-then-else문은 assingment를 수행할 때 사용됨

Pascal : if (count =0) then average := 0;
 else average := sum/count;

C : average = (count==0) ? 0 : sum/count ; // 참이면 0, 거짓이면 sum/count를 반환

#연산자 오버로딩

한 연산자가 다른 용도로 사용되는 경우. 남용하면 가독성과 신뢰성에 악영향.

+ 연산자 : 1 + 3 , 1.0 + 3.0 정수 더하기 연산자 || 실수 더하기 연산자 <- 오버로
 (FORTRAN) AVG = SUM / COUNT

float f ; int i=3, j=2; f = i / j // f는? -> 1.0이다! 컴퓨터는 정수와 실수 구분
 & 연산자 : c = a & b // 비트 AND 연산
 (C언어) c = &b // 주소 반환

/ 와 div 연산자(Pascal) : 피연산자가 실수면 '/'
 피연산자가 정수면 'div'

사용자 정의 오버로딩(Ada, C++) : 사용자가 정의한대로 연산을 수행할 수 있다.

A * B + C * D // 2차원 배열 Matrix 데이터타입의 행렬 연산으로 프로그래머가 정의

#타입 변환 (Type Conversion)

다른 타입끼리의 연산을 하는 연산자가 없기 때문에 강제로 형변환을 함

Coercion : 컴파일러에 의해 타입 변환이 묵시적으로 됨

Casting : 프로그래머에 의해 타입 변환이 명시적으로 됨

```

int a = 10;
double b = 10.0;
float c;
c = (double)a + b;            //coercion 1번(a+b인 double을 float로 형변환)            casting 1번(a를 double로 형변환)

```

타입 변환의 종류

Narrowing Conversion(축소 변환) : 원래 형의 모든 값을 포함할 수 없는 타입으로 변환

double -> float

Widening Conversion(확장 변환) : 원래 형의 모든 값을 포함할 수 있는 타입으로 변환

float -> double

축소 변환은 문제가 생길 거 같은데... 확장 변환은 안전한가? -> **확장 변환도 문제가 생길 수 있음★**

e.g. int -> float : 정수에서 소수로 바꾸는 것인데 표현할 수 있는 10진수 유효범위가 줄어듦.

QQ) 이게 확장변환인가?

int는 9자리, float는 7자리

-> 데이터 손실이 일어날 수 있음!

언어별 타입 변환

FORTRAN77 : int, real, double, complex

항상 확장 변환

C : short, int, long, float, double

대부분 확장변환, 축소변환도 일부 있음.

작은 범위로 수행 가능한 연산일지라도, 실제 연산은 **확장변환**해서 하는 경우가 있음.

e.g. 3.7+1.3 <- float로 할 수 있지만 실제로는 double로 처리함

e.g. 1+3 <- short로 할 수 있지만 실제로는 int로 처리함

Coercion에서의 문제

FORTRAN77에서는 매개변수 전달 시 **타입검사X**

A+D를 K에 매개변수로 전달해도 에러X

```
INTEGER A,B,C
REAL D
...
C = FUN(A+D)
```

```
function FUN (K:INTEGER) {
...
}
```

ADA/Modula2에서는 int와 float의 혼합된 연산 자체를 허용하지 않음.

Casting (Explicit Conversion)

명시적 형변환

ADA/Modula2 : 함수 형태로 구현

AVG := FLOAT(SUM) / FLOAT(COUNT);

C :

AVG = (int)SUM;

Casting에서 발생할 수 있는 오류들

Overflow :

Underflow :

double a; a=1.0e20+5-1.0e20 // 1.0의 10의20승

//너무 큰 수 사이에 작은 수를 연산하면 묻혀버림. 답이 0.0이 나옴.

//큰 수든 작은 수든 **부동소수점의 정밀도 문제** 때문에 발생

Divide by Zero : 0으로 나누면 예외 발생

#관계 표현

2개의 피연산자, 하나의 관계 연산자로 이루어져 있음

두 피 연산자를 비교하는 연산

관계 연산의 값은 Boolean으로 나옴!

관계 연산자는 다양한 타입에 대해 **오버로딩** 되어있음. ; 피연산자의 타입에 따라 계산이 달라짐

★관계 연산이 산술 연산보다 **우선순위가 낮다!**★

z + 1 > 2 * b ; * + > 순으로 연산

#Boolean 표현

boolean 변수, 상수, 관계 연산, 연산자로 이루어져 있음

Boolean 연산자 : AND, OR, NOT

>> Boolean 연산자끼리도 연산자 우선순위가 있다!

FORTRAN 77의 예

A + B .GT. 2 * C .AND. K .NE. 0

산술(* +)먼저 -> boolean(.GT. .NE. .AND.) 순으로

C에서는 Boolean type이 없음! 0이면 거짓, 0이 아닌 값이면 참 ==> 에러를 찾기 어려움

C에서의 문제 : a>b>c에서 a=10, b=5, c=3라고 할 때, a>b의 값이 True가 아닌 1이 나와 다음에 1>c 연산을 하고, 거짓으로 결과를 출력함

<-> Java에서는 True>3 자체에서 컴파일 에러를 냄.

#Short-Circuit Evaluation(단락 평가)

식의 일부분만 가지고 식 전체의 값을 미리 알 수 있는가?

어떤 언어가 허용하면 -> 식 일부만으로도 식 전체의 값을 미리 알 수 있다.

어떤 언어가 불허하면 -> 식 전체를 계산해야 한다.

(13 * A) * (B / 13 - 1)

//만약 13*A가 0이라면

뒤 값과 상관없이 값이 0

(A >= 0) and B < 0

//만약 A>=0가 거짓이라면

뒤 값과 상관없이 값이 거짓

어떤 언어가 단락 평가를 허용하면 뒷 부분 무시, 불허하면 뒷 부분도 계산

★수업에서는 논리 연산에 관한 단락 평가만 다룰 것!

Pascal (단락평가 불허 - 무시할 수 있을때도 뒷부분 실행)

10개짜리 리스트에 key라는 값이 들어있는지 확인하는 코드

```
int list[10];
```

```
int listlen = 10;
```

```
index := 1;
```

```
while (index <= listlen) and (list[index] <> key) do
```

//<>는 != Not Equal

```
    index := index + 1;
```

index가 10이 됐을 때도 and가 참이었으면 조건을 실행해 index가 11이 됨

그 다음 반복문에서는 index <= listlen이 11<=10이므로 거짓이지만 단락평가를 불허하므로 list[11]<>key도 실행. 이 때 list[11]은 없으니 out of index 에러(런타임 에러)가 남!

FORTRAN (단락평가 허용 - 무시할 수 있으면 무시)

(a > b) || (b++ / 3)

이 식이 실행될 때마다 b의 값을 늘리고 싶은데 a > b가 참이면 뒷 식이 무시돼 b++이 되지 않음

Ada (단락평가 할 수도 있고 안할수도 있음)

AND, OR(단락평가 불허) 이 있고 and_then, or_else(단락평가 허용)가 있음.

사용자에게 선택지를 줌!

```
index := 1;
```

```
while (index <= listlen) and_then (list[index] != key)
```

//<>는 != Not Equal

```
    loop
```

```
        index := index + 1;
```

```
    end loop;
```

C(단락평가 허용) : AND, OR

#배정문(Assignment Statement)

변수의 값에 대한 바인딩을 동적으로 변경할 수 있는 매커니즘.

배정문의 기본 형태 : <target_variable> <assignment_operator> <expression>

대부분 언어에서는 '='를 배정 기호(Assignment operator)로 사용

PL/1 에서는 배정과 '같다'가 동일해서 알아보기 힘들

ALGOL60에서는 ':=' 사용

동시에 여러 개 배정하기

PL/1 - SUM, TOTAL = 0

두 개에 값에 0을 assign

C언어는 동시에 안됨.

SUM=TOTAL=0

조건 타겟(Conditional Target)

C++ flag ? count1 : count2 = 0 ; // flag가 참이면 count1=0을 하고 아니면 count2=0

sum = flag ? count1 : count2; // flag가 참이면 sum=count1을 하고 아니면 sum=count2

복합 배정연산자(Compound assignment operators)

좌변과 우변의 공통된 피연산자를 축약

+=, -=, *=, /=

단항 배정연산자(Unary assignment operators)

++, --

prefix sum = ++count; //증가시키고 식을 실행하라

postfix sum = count++; // 식을 실행하고 증가시켜라

피연산자로서의 배정문

C 계열언어에서는 배정문의 결과를 피연산자로 사용할 수 있다.

```
while ( (ch = get()) != EOF) { ..... }  
/= while ( ch = get() != EOF) { ..... } // 대입연산자의 우선순위는 거의 제일낮음
```

sum = count = 0; //이것도 배정문의 결과가 피연산자로 사용되어서 가능함

따라서 if (x=y) 같은 구문도 에러를 일으키지 않고 작동함 -> 컴파일러가 에러를 찾지 못함

#배정문 안에 여러 개의 데이터 타입이 있을 때 배정 (Mixed-Mode Assignment)

표현식의 데이터 타입이 같아야 할까? 타입이 일치하지 않으면 coercion을 사용해야 하는가?

Fortran, C : 동일한 coercion 규칙이 있음. 여러 개의 데이터 타입이 있으면 **Coercion함**. 기본적으로 큰걸로

Pascal : 할당 등 일부 경우에 정수를 실수로 coercion함(제한적으로 coercion)

Ada, Modula-2, Java : coercion을 허용하지 않음

```
int i;
```

```
float a,b
```

```
i = a * i;
```

8장 - Statement-Level Control Structures 문장 간의 제어

순서, 선택, 반복문은 계산을 표현하는데 필수적

#개요

명령형 언어에서의 계산은 **표현들을 평가하는 것과 결과값을 변수에 할당하는 것**

명령형 언어에서의 제어는 선택과 반복이 있다.

초기 FORTRAN에서는 제어문이 기계 종속적이었다. -> 현대는 언어 안에서 디자인되어야 함
flowchart에서 표현되는 모든 알고리즘은 두가지 제어문으로 표현할 수 있다. 선택과 반복

제어문의 수가 커질수록 **작성력(Writability)**이 높아진다

제어문의 수가 커질수록 **단순성(Simplicity)**이 낮아지고, **가독성(Readability)**이 낮아진다.

-> 가독성을 낮추지 않고 작성력을 높이는 방법을 찾아야 함.

-> 또 제어문이 너무 적으면 같은 코드를 반복해서 써야 할 수도 있어 일정 개수 이상 쓰는 것이 바람직

e.g. C언어의 Switch문

순서도

순서도에서 마름모는 분기

Single-Entry, Single-Exit 제어문을 설계할 때는 한 개의 진입점과 한 개의 출구가 있어야 올바른 설계

#복합문과 블록

복합문(Compound statement) : 한 개의 문장으로 추상화될 수 있는 문장들의 묶음

블록(Block) : 복합문 + 변수 선언

ALGOL 60 : 복합문과 블록 사용

Pascal : 복합문 O, 블록 X

C : 복합문과 블록 둘 다 중괄호 { } 로 사용

제어문을 설계할 때 **여러 개의 진입점**이 있다면..? -> 복잡성을 증가시켜 **가독성을 감소**시킴.

그에 비해 유연성이 많이 증가되지는 않아 비효율적

#조건문(Selection Statement)

두 개 이상의 경로 가운데 하나를 선택하는 것.

양갈래 선택, n개 중 선택(다중 선택)이 있음.

#양갈래 조건문(Two-way)

디자인 이슈

조건을 제어하는 표현식의 형태와 타입은 어떻게 되는가?

대부분의 언어에서는 Boolean 표현식

C에서는 산술식 e.g. if(1)

조건이 맞았을 때 수행할 문장이 **한 문장만** 가능한가? **문장의 sequence**도 가능한가? **복합문**도 가능한가?

선택문이 중첩된 경우 then(else) clauses가 어디에 매치되는가?

단방향 선택문(Single-Way)

선택지가 하나밖에 없는 선택문(if만 있는 선택문). two-way의 하위호환.

FORTAN : IF(boolean 식) statement. if 다음에 **무조건 한 개의 문장만** 올 수 있음! 중첩도 불가능.

따라서 해결책으로 goto를 사용. 매우 간단하고 매우 유연하지 않음.

ALGOL60 : 복합문을 지원함. if 다음 begin end.

ALGOL이후 FORTRAN77, 90 등 단방향 선택문을 지원하는 언어는 복합문이나 sequence of statements 지원

양방향 선택문(Two-Way)

선택될 수 있는 두 개의 경로가 있음

ALGOL60 : if then{ } else{ } //안에 복합문이 올 수 있음

중첩 조건문(선택문)

if를 중첩해서 사용하면 모호함이 발생할 수 있음!

```
if(sum = 0) then
    if(count = 0)
        then result := 0
    else result := 1
```

대부분의 언어에서 else는 가장 최근에 선언된 then에 묶인다.

ALGOL60 : 모호함을 없애기 위해 복합문을 사용. 같은 begin end 안에 있어야 else로 묶임.

특수어(special word)를 사용해서 모호함을 줄일 수 있음

Modula-2 : END

FORTRAN77 : END IF

```
if (sum = 0) then
    begin
        if (count = 0)
            then result := 0
        else result := 1
    end
```

```
if (sum = 0) then
    begin
        if (count = 0)
            then result := 0
        end
    else result := 1
```

```
IF sum = 0
    THEN IF count = 0
        THEN result := 0
    ELSE result := 1
END
```

```
IF sum = 0
    THEN IF count = 0
        THEN result := 0
    END
    ELSE result := 1
END
```

#다중 조건문(Multiple selection)

여러 개의 문장이나 문장 그룹 중에서 하나를 선택하는 것

디자인 이슈

조건을 제어하는 표현식의 형태와 타입은 어떻게 되는가?

조건이 맞았을 때 수행할 문장이 한 문장만 가능한가? 문장의 sequence도 가능한가? 복합문도 가능한가?

다중 선택 구조가 하나의 문법적 구조로 완전히 캡슐화되어야 하는가? e.g. switch/break문

실행 흐름이 구조 내에서 하나의 선택가능한 명령문만 실행되도록 제한해야 하는가? 아니면 여러 선택지?

표현되지 않은 선택지가 있다면 어떻게 처리해야 할까?

FORTRAN : goto를 사용해서 다중 선택을 구현

ALGOL-W : 구조가 캡슐화되어 있으며, 각각의 segment가 하나씩 구분되어 있음. 하나만 실행하고 종료.

C언어의 switch와 유사. 하지만 break 없으면 계속실행하는 C와 달리 하나 실행하고 종료.

Pascal : 선택가능한 segment에 label이 나와 있음. 순서타입(Ordinal type; int, bool, char, enum)으로 이루어짐.

조건식을 먼저 실행하고, 상수 리스트의 상수와 비교를 함.

해당 segment가 있는 곳으로 가 실행함.

segment들이 상호배타적이어야 함. but 완전할 필요(전체를 커버할 필요)는 없음.

묵시적 분기(implicit branching) : 하나의 segment를 실행하면 그 조건문 나오는 것.

C : switch. 제어문과 상수 표현이 int 타입이어야 함.

implicit branching을 제공하지 않음. 한 segment가 실행되더라도 break가 없으면 나가지지 않음.
안정성(reliability)을 포기하고 유연성(flexibility)를 선택함.

Boolean 표현식으로 선택문을 만들 경우, 중첩 양갈래 선택문을 사용해 다중 선택문을 만들 수 있다.
-> if-elseif로 switch와 비슷하게 만들 수 있다.

#반복문 (Iterative statement)

0번, 1번, 또는 그 이상 구문을 실행하는 것

함수형 언어에서는 반복 구조보다는 재귀를 많이 사용함.

과거에는 기계적으로 반복문을 구현했지만, 현대는 언어 자체의 반복문으로 동작

디자인 이슈

어떻게 반복이 제어되는가? - 논리적? 카운팅? 아니면 둘을 같이?

반복을 언제 검사할 것인가? - pretest, posttest, 아니면 user-define

while do while break, continue

#카운터 제어 반복문 (Counter-controlled loops)

루프 변수 : 카운트 값을 유지한다. 실제로 세는 변수. 반복문에서 i 같은 변수

루프 매개변수 : initial(초기값), terminal(종료값), stepsize(스텝)

★루프 변수는 대부분의 경우 눈으로 볼 수 없다.

```
DO 30 I=1,100,2  
...  
30 CONTINUE
```

디자인 이슈

루프 변수와 매개변수의 타입과 스코프(그 변수를 참조가능한 범위)는 어떤 것이어야 하나?

int? char? enum? float?

c언어는 실수도 되고 char도 된다.

루프 변수가 끝난 다음 루프 변수는 어떤 값을 가져야 하나?

루프 변수나 루프 매개변수를 루프 안에서 변경할 수 있나?

c언어는 가능.

루프의 완료를 위에서 확인할 것인가? 아래서 확인할 것인가?

루프 매개변수를 한번만 평가할 것인가? 매번 평가할 것인가?

FORTRAN4 DO

posttest(후검사)

초기, 종료, 스텝 매개변수가 양수인 unsigned 정수 상수이거나, 간단한 정수 변수여야만 함.

루프 변수가 정상적으로 종료되면 undefined(쓰레기값), 비정상적으로 종료되면 가장 최근 할당된 값.

루프 안에서(body에서) 루프 변수나 매개변수의 값을 변경할 수 없음. -> 한번 반복 횟수가 결정되면 고정.

2개이상 문장 실행하려면 goto 사용

-> goto 때문에 do 문(반복문)을 거치지 않고 반복문 내부로 진입할 수 있는 문제 있음. (Single Entry X)

FORTRAN 77/90 DO

pretest(전검사)

루프 변수가 정수, 실수(real, double-precision) 가능.

루프 매개변수를 표현식(expression)으로 표현할 수 있었음. 양수 음수 모두 가능.

반복이 iteration count에 의해 제어. NOT 루프 변수. 따라서 루프 변수를 바꿀수는 있지만 반복에 영향X

iteration count는 내부 변수(internal variable)이어서 사용자가 접근할 수 없음.
DO 문을 거쳐야만 반복문을 실행할 수 있었음(Single Entry O)

ALGOL 60 for

카운터와 boolean 표현식이 합쳐져 반복문을 제어

```
for count := 1,2,3,4,5,6,7,8,9,10 do
  list[count] := 0
```

```
for count := 1 step 1 until 10 do
  list[count] := 0
```

```
for count := 1, count+1 while (count <= 10) do
  list[count] := 0
```

```
for index := 1,4,13,
  41 step 2 until 47,
  3*index while index < 1000,
  34,2,-24 do
  sum := sum + index
```

index

```
1, 4, 13,
41, 43, 45, 47,
141, 423,
34, 2, -24
```

for에 있는 모든 표현식은 매 반복때마다 평가됨.
C언어도 유사!

```
i := 1
for count := 1 step count until (3*i) do
  i := i + 1
```

can change step size

count	i*3
1	3
2	6
4	9
8	12
16	15

break

ADA for

pretest.

파이썬과 유사. in이 오고

reverse 옵션 : reverse를 쓰면 반대로 loop를 돌.

loop부터 loop end까지가 scope. 루프 안의 루프 변수는 독립적으로 작동. 따라서 아래 코드에서 루프가 끝나도 COUNT는 1.35로 유지.

루프 내부에서 루프 변수의 값을 변경시킬 수 없음.

```
COUNT : FLOAT := 1.35
for COUNT in 1..10 loop
  SUM := SUM + COUNT ;
end loop
```

```
for variable in [reverse] discrete_range loop
  ....
end loop
```

C for

pretest

Evaluate only once **Evaluate before each execution of body** **Evaluate after each execution of body**

```
for (expression-1) expression-2 expression-3
  statement
```

C언어는 boolean 타입이 없어서 조건식(expression-2)가 0이 되면 작동이 끝남. 아니면 계속 작동.

모든 expression들이 전부 선택적! 써도 되고 안써도 됨. for(;;)도 가능.

C언어는 루프 변수가 존재하지 않음. 루프 매개변수로가 count.

#논리 제어 반복문 (logical controlled loops)

- ★모든 카운터 제어 반복문은 논리 제어 반복문으로 만들 수 있다.
- ★모든 for문은 while문으로 바꿀 수 있다.
- ★역은 성립하지 않는다.

디자인 이슈

pretest? posttest?

카운터 제어의 특별한 형태인가? 아니면 완전히 다른(독자적인) 구문을 쓰는가?

몇몇 언어는 pretest와 posttest 둘 다 지원한다. - Pascal, Moudla-2, C 등...

C : Pretest - while Posttest - do while

Do while은 조건에 상관없이 최소 한번은 실행된다.

#사용자 제어 메커니즘 (User-Located loop control mechanism)

처음이나 끝이 아닌 곳에서 프로그래머가 루프 제어를 할 수 있게 한 것

디자인 이슈

조건문이 종료 매커니즘에 필수로 와야 하는가? (C언어는 break 이후로 조건문 안 옴)

한 조건문만 빠져나가는가? 아니면 감싸고 있는 다른 조건문도 빠져나가는가?

Ada exit

조건문이 있어서 조건을 만족시키면 빠져나가라

loop 마다 label이 있어서 안쪽 loop에서도 exit OUTER_LOOP when ~을 하면 한번에 빠져나갈수 있음

C break

continue와 break가 있음. 조건이 필요 없음.

중첩되어 있을 때 가장 안쪽에 있는 루프 하나만 나감.

In Ada,

loop can be labeled

```
OUTER_LOOP :  
  for ROW in 1..MAX_ROWS loop  
    INNER_LOOP :  
      for COL in 1..MAX_COLS loop  
        SUM := SUM + MAT(ROW, COL);  
        exit OUTER_LOOP when SUM > 100.0  
      end loop INNER_LOOP  
    end loop OUTER_LOOP
```

infinite loop

```
loop  
  ...  
  SUM := SUM + INDEX ;  
  exit when sum >= 1000;  
  ...  
end loop
```

In C,

skip the remaining loop body

```
while (sum < 1000) {  
  getnext(value) ;  
  if (value < 0) continue ;  
  sum = sum + value ;  
}
```

Multiple Exit

terminate the loop

```
while (sum < 1000) {  
  getnext(value) ;  
  if (value < 0) break ;  
  sum = sum + value ;  
}
```

#자료구조에 기반한 반복

어떤 특정한 자료구조에 기반한 리스트의 반복

파이썬 for for x in [1,2,3] #리스트에 기반한 for문

Java for for(String myElement : myList)

C# foreach foreach(String name in strList) ...;

#무조건 분기문 (Unconditional Branch)

특정한 장소로 제어를 넘겨준다.

goto문이 대표적.

무조건 분기의 문제점

goto - 가장 강력한 구문. 매우 강력해서 위험하다

위에서 아래로 일 때(작성된 순서대로일 때) 가독성이 좋다. <-> goto문은 가독성을 해친다.

몇몇 언어에서는 goto를 포함하고 있지 않음.

대부분의 현대 언어는 goto문을 포함하고 있다. - 하위 호환 때문

but, break와 같이 사용이 제한된 무조건 분기문은 가독성을 높일 수 있다!

Q) 무조건 분기문은 (항상) 가독성을 낮춘다? A) 아님. break 같은 것은 가독성을 높임.

Label Forms(레이블)

Goto문의 목적지를 지정하는 것

ALGOL60 & C : 식별자(identifier)를 레이블로 사용함

FORTRAN & Pascal : unsigned 정수 상수

PL/1 : 변수를 레이블로 사용하는 것 허용

레이블을 붙일 수 있는 장소의 제한

Pascal에서 '실행이 끝났거나', '실행 전인' 복합문(while 등) 안으로 goto를 할 수 없다.

```
while ... do begin
100:   ...
      ...
end
while ... do begin
      goto 100 ;
      goto 200 ;
end
while ... do begin
200:   ...
end
```

illegal

```
while ... do begin
100:   ...
      while ... do begin
            goto 100 ;
            ...
      end ;
end ;
```

legal

⇒ Pascal allows to branch to a different procedure (error-propagation)

#보호 명령(Guarded Commands)

선택문과 반복문을 다르게 사용하는 방식.

다이스트라(Dijkstra)의 선택 구조(Selection construct)

bool 식과 그것을 만족하면 실행하는 구문이 있음.

처음 시작할 때 모든 boolean 식을 평가함.

boolean 식 중에 여러 개가 참이면 -> 그 중 무작위로 실행.

하나가 참이면 -> 만족하는 것을 실행.

모두 거짓이면 -> 런타임 에러

```
if i = 0 -> sum := sum + i
[] i > j -> sum := sum + j
[] j > i -> sum := sum + I
fi
```

첫 번째와 세 번째 중 랜덤
if (i=0) and (j=1) ? 런타임 에러
if (i=j) and (i<>0) ?

다이스트라(Dijkstra)의 반복 구조(Loop construct)

처음 시작할 때 모든 boolean 식을 평가함.

boolean 식 중에 여러 개가 참이면 -> 그 중 무작위로 실행.

하나가 참이면 -> 만족하는 것을 실행.

모두 거짓이면 -> 반복이 종료

오름차순으로 정렬하는 코드

```
q1 := Q1; q2 := Q2; q3 := Q3; q4 := Q4;
do  q1 > q2 -> temp := q1; q1 := q2 ; q2 := temp ;
    [] q2 > q3 -> temp := q2; q2 := q3 ; q3 := temp ;
    [] q3 > q4 -> temp := q3; q3 := q4 ; q4 := temp ;
od
```

-> 동시성(concurrency) 제어를 위한 방법. 유용하다!

9장 - Statement-Level Control Structures 문장 간의 제어

서브프로그램은 프로그램의 기본 소요로서, 프로그래밍 언어 디자인에서 제일 중요한 부분 중 하나이다.

#Introduction

두 가지 추상화

프로세스 추상화(서브프로그램) : procedure call - 구문들의 집합을 추상화

데이터 추상화 : 데이터 타입의 추상화

현대 프로그래밍 언어에서 코드는 **재사용**된다.

메모리 공간 절약, 코딩하는 시간 절약

'call(호출)'을 함으로서 디테일한 부분을 추상화 할 수 있다.

caller(호출하는 것) vs callee(호출당하는 것)

Procedure vs Macro

Procedure : 복사가 되지 않고, 호출 시 어떤 값만 반환됨.

호출과 반환 O

Macro : 컴파일 시 그대로 코드가 복사됨.

호출과 반환 X

#Subprogram의 기본

SP의 기본 특징

Single Entry point 호출을 해서만 접근할 수 있음

부른(Calling) 프로그램은 SP이 실행중일 동안은 멈춰있다. **특정한 시간에는 하나의 SP만 실행이 가능.**

but, 동시성을 지원할 때(멀티스레드 프로그램)에서는 동시에 여러개의 SP가 실행될 수도 있음.

SP가 종료되면 제어권은 **항상 부른 곳(caller)**로 넘어간다.

기본 정의

SP 정의 : SP 추상화의 행동을 묘사하는 것.

SP 호출 : 명시적으로 SP가 실행되도록 요청하는 것

active(활성화) : SP가 호출되었지만, 아직 실행이 끝나지 않은 상태

SP의 종류

Procedure

매개변수화된 계산으로 정의된 구문의 집합

결과를 caller에게 두 가지 방법으로 전달한다.

1. visible한 비지역변수를 변경 2. 이전에 호출된 변수 값을 변경(Scope를 지원하는 언어에서)

리턴값이 없다.

함수(Function)

리턴값이 있다.

새로운 연산자를 정의할 수도 있음.

연산자 오버로딩 가능.

```
function power (base, exp : real) : real ;  
begin  
    .....  
end  
.....  
result := 3.4 * power(10.0, x)
```

SP의 헤더

뒤따르는 문법적 구조가 SP 정의임.

SP의 이름을 제공

선택적으로 매개변수를 지정할 수 있음.

Fortran SUBROUTINE ADDER (parameters)

Ada procedure ADDER (parameters) is

C C에서는 Function만 있다!

매개 변수(Parameter)

SP에서 데이터에 접근하는 방법에는 두 가지 방법이 있음

1. 비지역변수에 직접적으로 접근 ; 비지역변수의 접근은 reliability(신뢰성)을 낮춘다.
2. 매개변수 전달로

일부 경우에 SP를 매개변수로 사용할 수 있다.

형식 매개변수(Formal Parameter) vs 실매개변수(Actual Parameter)

형식 매개변수 : SP 헤더에 있는 매개변수 int n

실 매개변수 : SP가 호출될 때 실제 전달되는 매개변수 int i

```
void main(void) {
    int i, sum=0 ;

    for (i=1;i<1001;i++){
        sum=sum+factorial(i);
    }
}
```

```
int factorial(int n) {  
    if (n==1) return(1)  
    else return(n * factorial(n-1));  
}
```

매개변수의 전달

위치 매개변수 : 위치에 따라서 형식-실 매개변수를 맵핑. 첫 실매개는 첫 형식매개로 전달됨.

키워드 매개 변수 : 키워드에 따라서 어떤 실매개 변수로 전달될지 특정됨.

Ada에서는 사용자가 형식매개의 이름을 다 알아야 함! - 모르면 못씀.

C++/Ada에서는 형식매개변수가 기본값(default value)를 가질 수 있음. 실매개변수에서 지정을 안하면 기본값 전달.

실매개의 수는 형식매개의 수와 일치해야 한다.

-> C언어는 그 예외이다. e.g. `printf("%d %d", i, j);` `printf("%c", c);`

#SP의 다지인 이슈

어떤 매개변수전달 방식을 써야 하는가?

Pass-by-Value, Result, Value-Result, Reference, Name

형식 매개변수에 실 매개변수가 들어갈 때 타입 검사를 해야 하는가?

지역 변수가 동적/정적으로 할당되는가?

매개변수로 SP가 전달될 때 참조 환경은 어떻게 되는가?

SP를 매개변수로 전달할 때 타입 검사를 해야 하는가?

오버로딩(재정의)이 가능한가? 한번 정의한 함수를 또 정의할 수 있는가? - Java 가능 C 불가능

generic한가? - 다양한 데이터 타입에 대하여 쓸 수 있는가?

분리(seperate), 독립(independent) 컴파일 가능인가?

#지역 변수의 참조환경

지역변수 : SP 내부에서 선언된 변수

일반적으로 선언된 SP로 제한된다.

스택동적 지역변수

: SP가 실행되면 바운딩되고, 종료되면 해제되는 변수

장점 : 재귀를 허용함.

기억공간을 공유할 수 있음.(동시에 실행되지 않으면 다른 게 기억공간 사용)

단점 : 호출이 될 때마다 할당, 종료될때마다 해제, 비용이 들.

간접 참조(변수의 주소가 실행시간에 정해짐) -> 느리다.

history-sensitive(과거 기억)을 허용하지 않음. 해제되면 날아감.

정적 지역변수

: 프로그램이 시작될 때 기억공간에 바인딩 됨

장점 : 빠른 참조. history-sensitive함.

단점 : 재귀 허용X

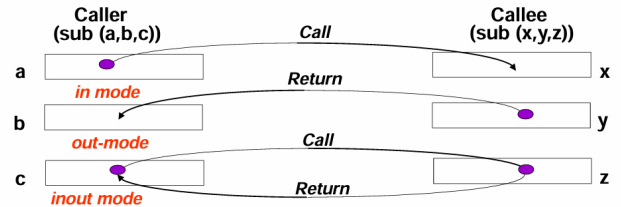
ALGOL 60과 후속 언어(C 등)들은 변수들이 기본적으로 스택동적으로 정의된다.

C에서는 static 키워드로 정적변수 선언 가능

FORTAN77에서는 모든 지역변수가 정적. 재귀 허용X

#매개변수 전달 방법

매개변수가 SP로 전달되는 방법들



매개변수 전달의 의미론적 모델

in mode : 형식 매개변수가 실 매개변수로부터 데이터를 받는 것

out mode : 형식 매개변수가 실 매개변수로 데이터를 보내는 것

in-out mode : in mode + out mode

어떻게 매개변수 전송이 실제로 어떻게 일어나는지에 대한 2가지 개념 모델

1. 실제 값이 물리적으로 옮겨짐
2. 접근 패스(포인터)가 옮겨짐

매개변수 전달의 구현 모델

: 3개의 전송 모델의 구현을 가이드하기 위해 다양한 모델이 개발되었다.

Pass-by-Value

in mode를 구현.

실매개가 형식매개로 전달되어 초기화되는데 사용됨.

주로 실제 데이터 전달(물리적)로 구현됨 - 큰 오브젝트를 옮길 때는 costly함. e.g. int a[100][100]

Pass-by-Result

out mode를 구현. caller에서 SP로 전달되는 값이 없다!

형식 매개가 SP 내에서 지역변수처럼 쓰임. 제어가 되돌아가기 직전에 caller의 실 매개로 전달.

제약조건 : 실 매개가 반드시 "변수"여야함! 상수이면 값을 전달 못 받기 때문.

형식 매개와 실 매개의 크기가 항상 동일해야 한다. 한쪽이 커지면 나머지도 커져야

단점 : 사이즈가 큰 변수 저장, 전달에 자원이 듬.

실 매개에 충돌이 발생할 수 있음. -----p1이 뭘까?--->

실 매개의 평가를 호출 시에 할지 반환 시에 할지에 따라서

값이 달라질 수 있다. => 코드 이식이 불가능해짐.

```
int index, list[10];
subroutine sub(a){
  index = 5;
  a = 3;
}
main() {
  index = 3;
  sub(list[index]);
}
```

index[3] or index[5] ?

```
subroutine sub(x,y){
  x=3 ;
  y=5;
}
main() {
  int p1;
  sub(p1, p1);
}
```

Pass-by-Value-Result (Pass-by-copy)

in-out mode를 구현.

처음에는 실 매개가 형식매개를 초기화하는데 쓰이고, 그 이후에서는 지역변수처럼 동작.

종료되면 형식매개에서 실 매개로 전달됨.

Pass-by-Reference

in-out mode로 구현.

pass-by-Copy와 차이는 값이 직접 복사되는 것이 아니고 접근 경로(Access path)가 전달이 됨.
주로 주소가 전달됨.

실 매개변수가 SP내에서 공유될 수 있다. // SP 안팎에서 접근할 수 있다.

값을 복사하는 것이 아니라 overhead가 없다. 공간 복사하는 것이 아님!

프로그램이 실행 중에 비정상적으로 종료되면 Pass-by-Copy는 매개변수가 바뀌지 않음

--> 함수가 정상적으로 끝나야만 값이 변한다

Pass-by-reference는 주소에 접근해 값을 그때그때 바꾸므로 실시간으로 매개변수가 바뀜.

단점 : 주소를 참조해서 값에 접근(indirect)하므로 느림

의도치 않게 실 매개변수의 값을 변하게 할 수 있음.

alias가 만들어질 수 있음. -> 있으면 안 좋음!

alias의 예시

```
procedure bigsub ;  
  var global : integer ;  
  procedure smallsub (var local : integer) ;  
  begin  
    global = 3; local = 5;  
  end  
begin  
  smallsub(global) ;  
end
```

```
procedure sub (var first, second : integer)  
...  
call sub (total, total) ;  
...
```

Pass-by-Name

in-out mode로 구현.

코드가 그대로 복사됨! 텍스트로 대체됨. 실매개변수가 형식매개변수로 대체.

실매개변수 형식매개변수 간에 어떠한 데이터 전달도 일어나지 않는다!

late binding : 실제로 형식 매개변수로 지정되거나 참조돼 봐야 어떤 동작을 할 수 있는지 알게 된다.

실매개변수의 타입에 따라서 어떤 방식으로 구현할지가 달라짐.

변수를 넣으면? -> P-b-reference처럼 구현

상수를 넣으면? -> P-b-value처럼 구현

배열/변수가 있는 표현식을 넣으면? -> 그때그때 값이 달라짐!

엄청난 유연성을 제공하지만, 프로세스가 느리고, 구현하기가 어려움.

읽는 사람과 쓰는 사람 모두 헷갈릴 수 있음.

```
procedure swap (FIRST, SECOND)  
  integer FIRST, SECOND ;  
  begin  
    integer TEMP ;  
    TEMP := FIRST ;  
    FIRST := SECOND ;  
    SECOND := TEMP  
  end ;
```

Pass-by-name

```
swap (KK, II) ;  
  
  TEMP = KK ;  
  KK = II ;  
  II = TEMP ;
```

OK!

```
swap (I, A[I]) ;  
  
  TEMP := I ;  
  I := A[I] ;  
  A[I] = TEMP ;
```

OK?

A[A[I]] = TEMP

젠슨 장치(Jensen's Device)

pass-by-name을 사용해서 하나의 procedure가 다양한 목표로 쓰일 수 있게 하는 것.
식과 하나 이상의 변수를 매개변수로 전달하는 것

```
real procedure SUM (ADDER, INDEX, LENGTH) ;
real ADDER ;
integer INDEX, LENGTH ;
begin
  real TEMPSUM ;
  TEMPSUM := 0.0 ;
  for INDEX := 1 step 1 until LENGTH do
    TEMPSUM := TEMPSUM + ADDER ;
  SUM := TEMPSUM
end ;
```

$$\sum_{i=0}^{100} (A[i])^2$$

```
SUM (A, I, 100) -> 100*A
for I := 1 step 1 until 100 do
  TEMSUM := TEMPSUM + A ;
```

```
SUM (A[I]*A[I], I, 100)
for I := 1 step 1 until 100 do
  TEMSUM := TEMPSUM + A[I]*A[I] ;
```

```
SUM (A[I], I, 100) ->  $\sum_{i=0}^{100} A[i]$ 
for I := 1 step 1 until 100 do
  TEMSUM := TEMPSUM + A[I] ;
```

```
SUM (A[I]*B[I], I, 100)  $\sum_{i=0}^{100} (A[i]*B[i])$ 
for I := 1 step 1 until 100 do
  TEMSUM := TEMPSUM + A[I]*B[I] ;
```

ALGOL 60에서 처음으로 call-by-name 사용.
같은 형식 매개변수여도 그때그때 달라짐!

thunk : 형식 매개변수가 실제로 어떤 실 매개변수와 연결이 될지 그때그때 계산해야 하는데, 내부에서 대신 계산해주는 숨겨진 루틴.

실제로는 pass-by-name을 이용해서 프로그램을 짜기가 매우 어려움
thunk의 cost가 너무 컸다. ALGOL 68부터 call-by-reference로 대체됨

#주요 언어에서의 매개변수 전달 방법

FORTRAN

77이전은 p-b-reference 77이후는 p-b-value-result

ALGOL60 & SIMULA67 p-b-name

ALGOL68 & C p-b-value

포인터 타입을 지원. p-b-reference의 효과를 줌! >> 실제로 p-b-reference를 지원하는 것이 아님.
형식 매개변수에 형식을 직접 지정하였음. `proc adder = (ref int a, int b, ref real c)`

ALGOL W p-b-result

Pascal & Modula-2

기본적으로는 p-b-value, 'var'이라는 특수어를 변수 앞에 붙이면 p-b-reference로 전달.

```
procedure adder (var a : integer ; /* call-by-reference */
                 b : integer ; /* call-by-value */
                 var c : real) ; /* call-by-reference */
```

Ada

앞에 특수어를 out, in, inout을 직접 붙여서 사용.

```
procedure ADDER (A : in out INTEGER ;
                 B : in INTEGER ;
                 C : in out FLOAT) is
```

매개변수의 타입 검사

매개변수의 타입이나 개수에 대해서 검사를 하는가?

FORTAN77 : 매개변수 검사 안함

Pascal, Modula-2, FORTRAN90 : 매개변수 타입 검사 함.

Original C : 매개변수의 개수와 타입을 검사 안함(FORTAN77과 같음)

ANSI C : 2가지 방식을 제공함. Original C처럼 아무 검사도 안하는 방식
+ 프로토타입 방법(타입 검사와 coercion을 함.)

C99이상 C++ : 타입 검사를 생략할 수 있음.

e.g. `printf(const char*, ...)` //그때그때 들어오는게 다름

매개변수 전달의 구현

ALGOL 60과 그 이후 언어들은 매개변수가 선언될 때 스택에 쌓임. (@ 런타임)

Pass-by-value

실매개변수의 값이 stack으로 복사가 된다.

형식매개는 실매개와 크기가 동일해야 함.

Pass-by-Result

함수가 종료됐을 때, 제어권이 호출자로 넘어가기 전에, 형식매개의 값이 실매개로 복사됨.

형식매개는 실매개와 크기가 동일해야 함.

Pass-by-Value-Result

p-b-value + p-b-result

형식매개는 실매개와 크기가 동일해야 함.

Pass-by-Reference

형식매개와 실매개는 다를 수 있다. 작을 수도 있고 클 수도 있다.

형식매개는 “주소”가 전달되므로 타입이 달라도 항상 동일

실매개에 대한 평가를 해야 함. -> `call sub(a*b)` // a*b한 값이 실매개이면 a*b한 값을 어딘가 저장하고 위치 포인터를 전달해야 함.

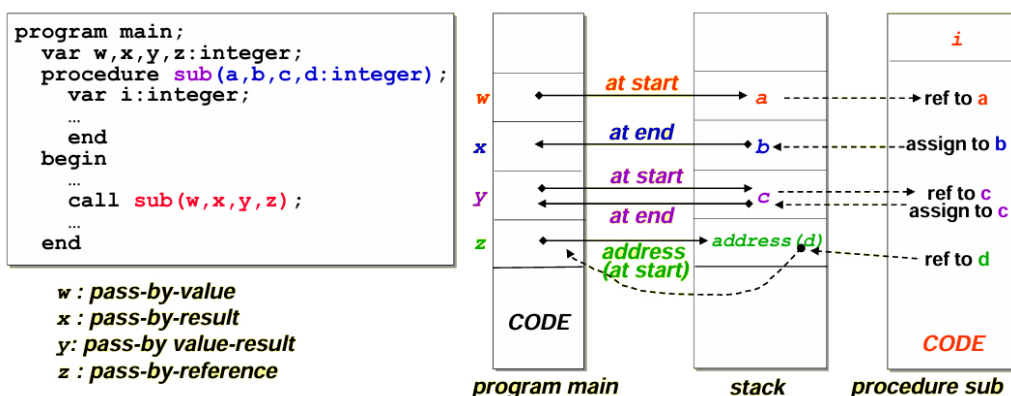
Pass-by-Name

thunk(비용이 많이 듦) : 파라미터 없는 procedure로 구현하거나,

컴파일 시 코드를 집어넣는다(run-time-resident)

모든 pass-by-name 매개변수에 대해서 각각 thunk가 호출되어야 한다.

어떤 형식 매개가 어떤 실매개에 연관이 있는지 파악하고, 그에 따른 주소를 연결해 줌.



Pass-by-Value-Result와 Pass-by-Reference의 차이 : V-R은 처음 호출됐을 때, 종료 직전 값이 전달되는 반면, Ref는 실시간으로 값이 변경됨.

디자인 고려사항

효율성

일방향, 양방향 데이터 전송

SP에 의한 SP 외부의 데이터(비지역)를 변경하는 것은 최소화 해야함. in-mode를 지향하는 것이 바람직.

SP이름을 매개변수로

SP 이름이 매개변수로 다른 SP로 전달되는 경우

SP 매개변수에 대한 설명이 SP이름과 함께 전달이 되어야 한다.(타입검사를 위해서)

3가지 바인딩 방법

Shallow binding : 전달된 SP를 실행시키는 부른 곳(함수)의 환경

SUB4에서 나를 불렀으니 x=4이다.

Dynamic scope 언어에서 주로 사용

Deep binding : 전달된 SP의 정의환경. 구조를 봄!

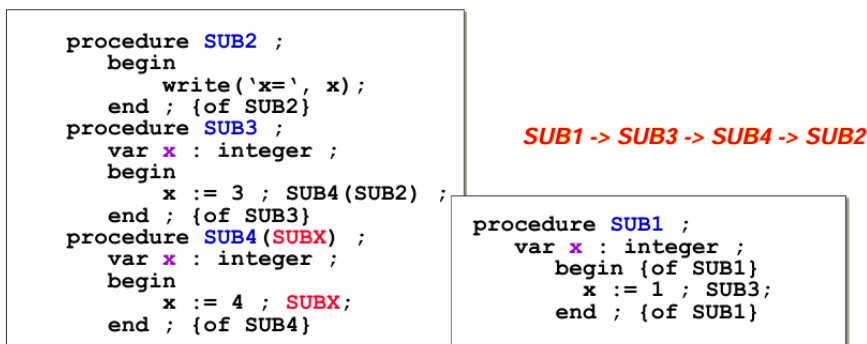
현재 계층에서 구조적으로 내 상위는 누구인가?

구조적으로 상위는 SUB1이므로 x=1

Static scope 언어에서 주로 사용

Ad hoc binding : 내가 '실매개로서 호출되기 전', 가장 최근에 할당된 x의 값은 무엇인가?

SUB2는 SUB4(SUB2)로서 이때 실매개로서 호출이 되었다. 이 때의 환경은 SUB3이다 따라서 x=3



SUB1이 나머지 procedure보다 한 칸 상위에 있음!

SP의 오버로딩 (Overloaded SP)

SP의 이름이 같지만 매개변수가 타입과 반환값은 unique해야 함.

실제로 어떤 걸 호출할지는 실매개의 목록을 가지고 결정!

Ada에서는 함수와 procedure 모두 오버로딩을 지원

C++에서는 형식매개변수에 기본값을 설정할 수 있음. 안넣으면 기본값으로 초기화됨.

but 모호한 경우가 발생할 수 있다. 모호하면 에러 남.'

```
void fun(int a){}
```

```
void fun(double b){}
```

```
fun(10);
```

//어디로 가야할까? coercion을 하면

//헛갈리므로 이런 경우 coercion 불허

```
void fun(float b = 0.0) {
  ...
}
void fun() {
  ...
}
main() {
  ... fun() ; /* ?? */
  ...
}
```

에러남.

Generic SP (제너릭 SP)

Generic : 다양한 데이터타입에 대응할 수 있는 것. 일반적.
프로그램의 재사용성을 높을 수 있음.

코드에서 선언이 되어야만(어떤 타입인지 명시를 해야만) 컴파일러가 포함시킴. 아니면 포함X
어떤 특정한 타입의 인스턴스가 생기지 않는 한 전체 프로그램에 영향을 미치지 않음.
C++의 템플릿, Java <T>

분리(Seperate) 컴파일, 독립(Independent) 컴파일

큰 SW시스템에서는 통째로 컴파일하기가 힘들. 나눠서!

컴파일 유닛(Compilation unit) : 컴파일 할 수 있는 최소단위

분리 컴파일(Seperate) : 매개변수의 타입 검사 O

컴파일 유닛끼리는 서로 다른 시간에 컴파일이 가능. 하지만 서로 호출이나 사용을 할 수 있기 때문에 독립적이지는 못함(검사해야 하기 때문). 반드시 다른 유닛도 참고해야 함.

컴파일러가 각 유닛이 어떤 정보(프로그램 entity; 변수, 타입, SP 등등)를 가지고 있는지를 알고 있어야 함 (접근할 수 있어야 함).

Ada에서는 컴파일러가 읽을 수 있는 library에 entity들을 저장해 컴파일 할 때 컴파일러가 참조할 수 있게 함
Ada, Modula-2, FORTRAN 90

독립 컴파일(Independent) : 매개변수의 타입 검사 X

컴파일러가 다른 프로그램 유닛 없이 컴파일을 할 수 있음.
컴파일 유닛끼리 서로 검사를 하지 않음.
일관성 검사X
FORTRAN77, C

함수에 대한 디자인 이슈

1. Side Effect를 허용할 것인가? ; 함수에서 비지역변수(전역, 다른 매개변수)를 변경시킬 것인가?
2. 어떤 타입의 값들이 몇 개 리턴될 것인가?
스크립트 언어(Ruby, Lua, Python)는 여러 개 리턴됨

함수의 Side Effect

Ada : 막기 위해서는 in-mode로 구현을 해야 함. 매개변수를 통해 Side effect를 막음. alias를 막을 수 있다.
Pascal, C : Side Effect를 허용함. alias가 발생할 수 있음
Pascal은 P-b-value와 p-b-reference 지원,
C는 p-b-value만 지원하지만, p-b-reference를 쓰는 것과 같은 효과를 볼 수 있음.

반환 값의 타입

Fortran77 : 구조화되지 않은 타입만(일반적인 타입) 반환 가능 <-> 구조화된 타입(union, array, record 등)
Pascal, Modula-2 : 간단한 타입만 지원. int, real, char, Boolean, pointer, enum
C : 대부분 타입가능. but 배열(array), 함수는 불가
Ada : 모두 가능

비지역변수에 접근하기

비지역변수 : 볼수는 있지만 지역에서 선언되지 않은 변수

Static scoping 언어 : 코드를 짤 때 코드의 구조를 보고 알 수 있음.

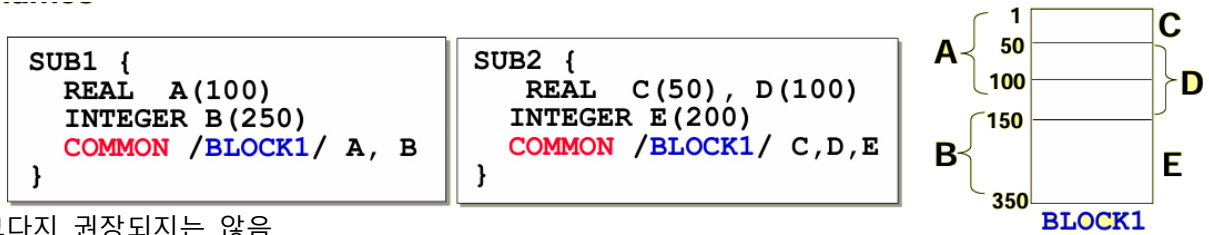
Dynamic scoping 언어 : 나를 호출한 곳의 scope를 참조 가능. 텍스트의 위치와는 무관.

정적으로 비지역변수에 대한 타입 검사를 할 수 없음 => 실행시간에만 가능

FORTRAN의 Common 블록

모든 SP에서 접근가능한 블록.

지역변수는 함수가 종료되면 사라지므로, 동시에 실행되지 않는다고 하면 여러 함수가 메모리 공유가능



그다지 권장되지는 않음

Modula-2, Ada

'with'라는 키워드를 사용해 어떤 모듈에 접근할 것인지 명시적으로 표현.

C

중첩을 허용하지 않음(Static scope, dynamic scope 등)

유일한 방법은 전역변수. 전역변수는 함수의 바깥에 정의.

extern이라는 접두사는 다른 파일에서 정의된 전역변수이다.(조건1 - 전역변수, 조건2 - static이면 안 됨)

사용자 지정 연산자 오버로딩

Ada, C++, Python에서는 사용자가 연산자를 오버로딩 할 수 있다.

e.g. 곱하기 연산자를 행렬곱으로 재정의할 수 있다.

10장 - Implementing Subprograms subprograms의 구현

언어가 어떻게 동작하는지 알 수 있음. SP의 구현이 어려워지는 이유는 재귀와 비지역변수의 접근 때문에 어려워짐.

Subprogram Linkage

SP를 **호출**하고 **반환**하는 과정을 linkage(연결)라고 함

linkage = 호출 + 반환

SP 호출의 동작

어떻게 전달할 것인가? parameter passing mechanism

SP의 변수들을 메모리에 어떻게 올릴 것인가?

현재 실행 상태는 어떤가? 활성화 상태인가?

활성화==호출은 되었지만 종료되지 않은 상태

제어권을 SP로 넘기고, 종료되었을 때 제대로된 곳으로 제어권을 돌려주는가(어디로)?

비지역변수의 접근을 허용하면 그 매커니즘은 어떻게 할 것인가?

SP 반환의 동작

out-mode이거나 in-out-mode일 때, 실매개변수로 어떻게 전달할 것인가?

지역변수에서 사용한 공간을 해제(deallocate)해 줘야 함.

비지역변수를 참고한 것도 해제해야 함.

제어권을 호출한 곳으로 돌려줘야 함.

“간단한” SP의 구현

1. SP에 **재귀가 없다**
2. 모든 비지역변수에 대한 참조는 COMMON에서만 한다.(비지역변수는 따로 모아둠)
3. 모든 변수가 **정적 변수**이다.

FORTRAN77은 간단한 SP이 구현되어 있음.

FORTRAN77에서의 호출

step 1. 현재 실행중인 프로그램 유닛의 상태를 저장

step 2. 매개변수 전달 과정 수행.

FORTRAN77은 p-b-copy

step 3. callee에게 제어권을 넘겨야 하는 주소를 넘겨줌(함수 종료 후 제어권 받을 곳)

step 4. 제어권을 callee에게 넘겨줌

FORTRAN77에서의 반환

pass-by-value-result라면, 실매개변수로 전달할 값을 호출자가 읽을 수 있는 위치로 옮겨줌.

SP가 함수라면, 리턴할 값을 호출자가 읽을 수 있는 위치로 옮겨줌.

호출자의 실행 상태를 복구함. SP가 종료되었으니까 원래 호출한 애를 다시 실행하게 함.

제어를 호출자에게 돌려줌.

어떤 기억공간이 필요할까?

호출자(caller)에 대한 상태 저장

매개변수

리턴 주소

함수라면 : 함수값을 저장하는 곳

저장공간은 크게 코드와 데이터로 나뉨.
 코드 부분은 한번 컴파일 되면 그 이후로는 바뀌지 않음. 크기도 고정
 데이터 부분은 **활성화 레코드(Activation Record)**라고 불림.

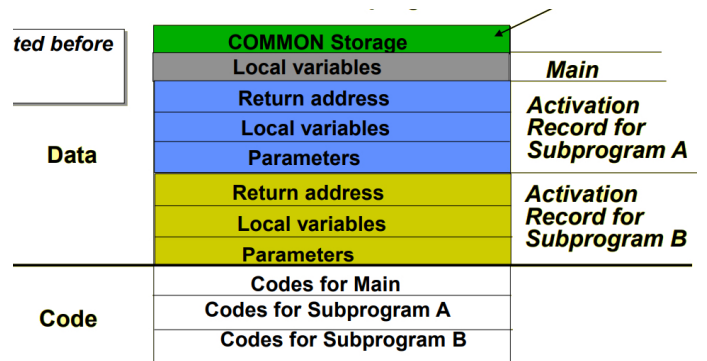
재귀가 있으면 하나의 프로그램이 여러 활성화 레코드를 가질 수 있음.
 재귀가 없으면 한 시점에 무조건 하나의 활성화 레코드를 가짐. >> 그래서 정적으로 할당될 수 있음.

FORTTRAN은 분리 컴파일이 가능. 따로 있는 것을 묶는 것 **Linker**. 하나의 활성화 레코드로 만드는 것 **Linking**

스택 동적 변수를 가지고 있을 때의 SP 구현

스택 동적 변수 : 함수의 생명주기동안만 있는 변수

ALGOL-like 언어는 FORTRAN77보다 '복잡함'



- 매개변수가 2가지 이상의 전달 방식을 사용함.
 e.g. Modula-2는 p-b-value와 p-b-reference 둘 다 사용
 전달 때마다 달라짐...
- 변수가 동적으로 할당됨.
- 재귀를 허용함. 하나의 SP이 여러 개의 활성화 레코드를 가질 수 있음.
- ALGOL-like 언어는 Static Scope를 사용함. 구조적 상위에 있는 변수 참조. 비지역변수를 참조하는데 어려움.

활성화 레코드의 생성

실행 도중에 **활성화 레코드 인스턴스(Activation Record Instance)**가 계속 만들어지고 변함. 동적임.
 마지막에 호출된 것이 마지막에 끝난다. 후입선출. 스택에 구현.
 >>모든 함수가 호출되면, 재귀이든 아니든 새로운 **활성화 레코드 인스턴스**를 스택에 생성함.

활성화 레코드

SP의 활성화 레코드의 형식과 크기는 컴파일 시간에 알 수 있음.

Dynamic 링크 : Dynamic scope를 구할 때에 사용 + 해당 SP의 활성화 레코드의 끝을 보여줌.

현재 **활성화 레코드 인스턴스**가 끝나면 여기까지 꺼내라!(파괴) 할 때 사용.

Static 링크 : Static scope를 구하기 위한 포인터. 누가 내 상위에 있는지를 가리키고 있음.'

Static scope를 구하기 위해서만 씀.

리턴 주소 : 끝났을 때 되돌아갈 곳 저장. (code_segment, offset)

지역 변수 : 지역 변수.

매개변수 : 매개변수로 받은 값들 저장. 실매개변수의 값 저장.

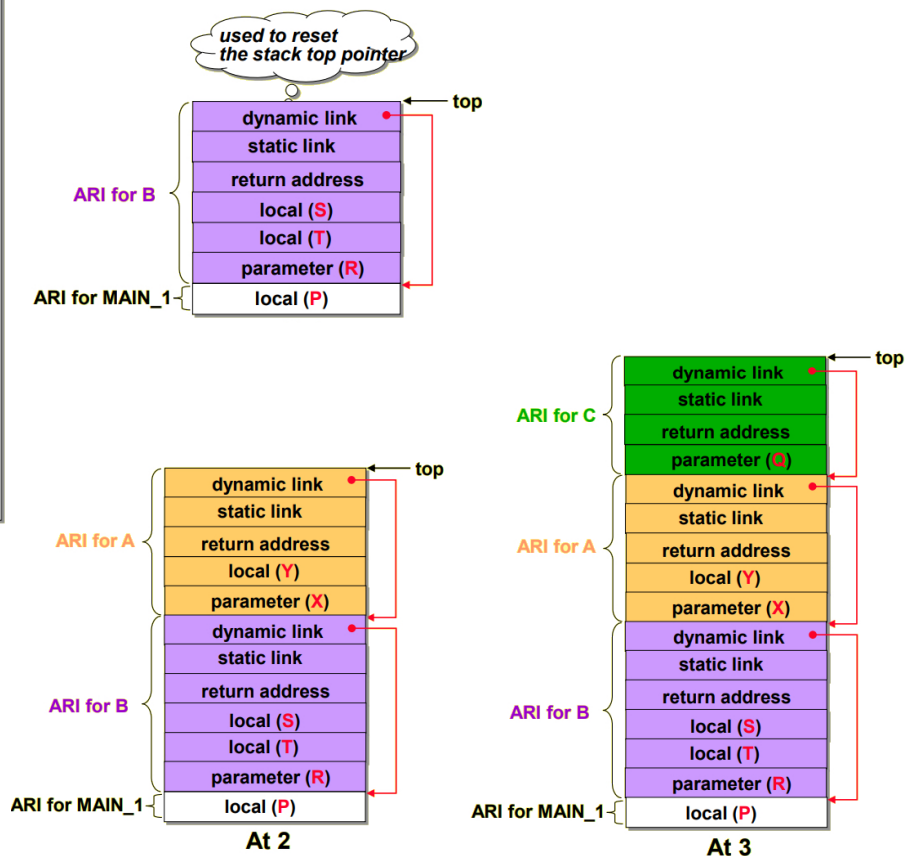
```

program MAIN_1
var P : real;
procedure A(X:integer);
var Y:boolean;
procedure C(Q:boolean);
begin
    P=P+1; ← 3
end
begin
    ... ← 2
    C(Y);
    ...
end {end of procedure A}
procedure B(R:real);
var S,T;
begin
    ... ← 1
    A(S);
    ...
end {(end of procedure B)}
begin {Main_1}
    ...
    B(P);
    ...
end

```

MAIN_1 -> B(P) -> A(S) -> C(Y)

매개변수는 형식매개변를 쓰는것!
NOT 실매개



Dynamic Chain : Dynamic 링크가 연결된 집합. 호출 순서를 나타냄. a.k.a. **Call Chain**

Local offset : 지역변수 참조 시에 사용하는 offset. 정수 숫자.

ARI의 시작 지점에서 얼마나 떨어져있는가?

컴파일 시간에 결정됨.

순서를 이용해서 선언된 변수의 타입, 크기 등을 알 수 있다.

재귀가 있는 예시

*procedure가 아니고 함수여서
function value가 추가됨*

• **Example : Using recursion to Compute the factorial**

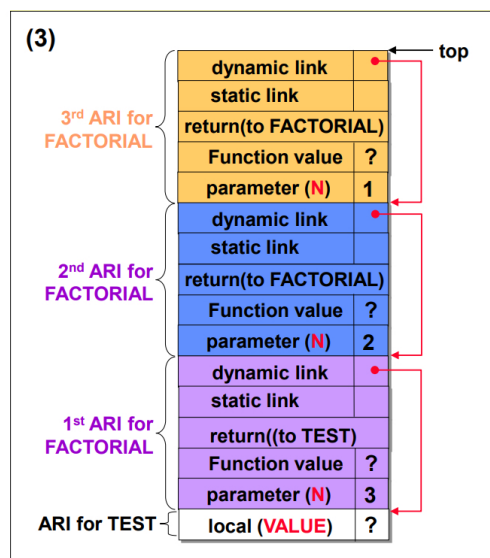
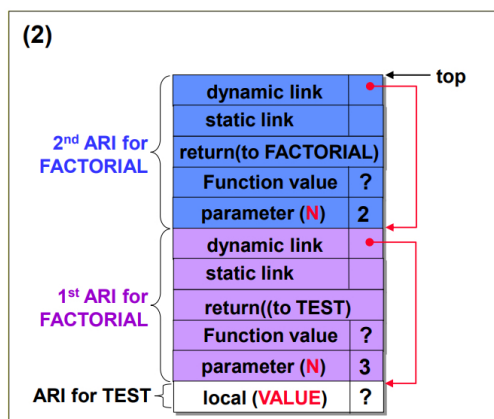
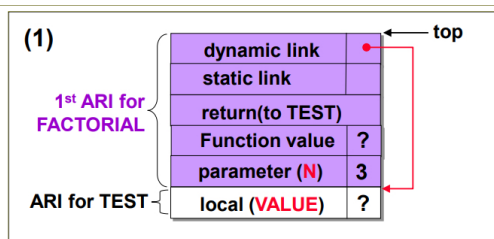
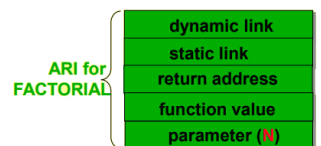
```

Program TEST
var VALUE:integer;

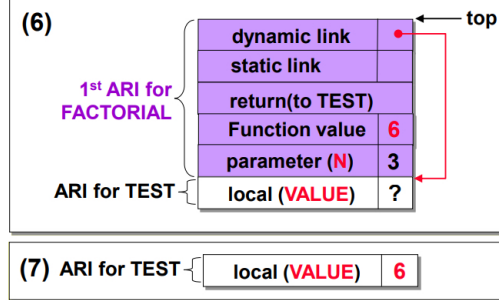
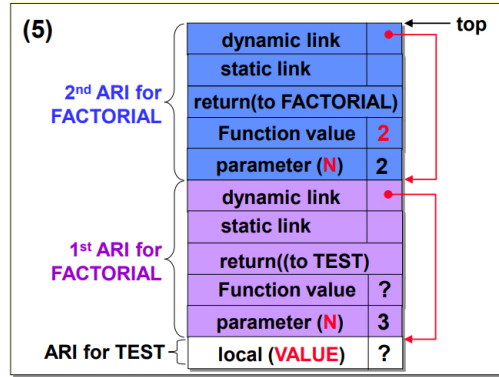
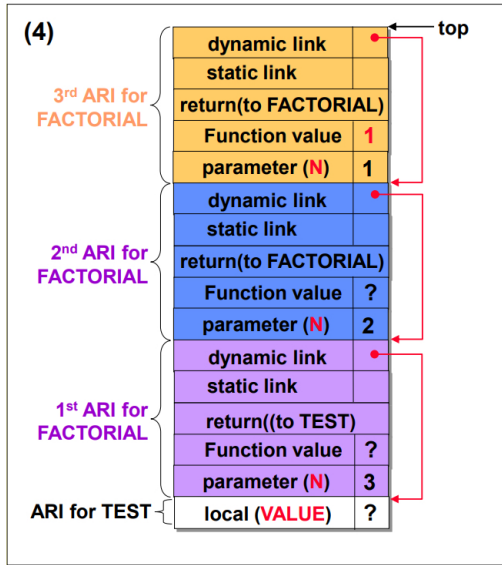
function FACTORIAL(N:integer);
begin
    if N<=1
    then FACTORIAL:=1
    else FACTORIAL:= N*FACTORIAL(N-1);
end

begin
    VALUE:=FACTORIAL(3);
    writeln("factorial 3 is:", VALUE)
end.

```



(1),(2),(3) 번은 1번 위치에서



(4),(5),(6)은 2번에서
(7)은 3번에서

SP의 중첩

중첩 시 Static scope를 지원하는 언어이면?
SP에서 비지역변수를 접근하는 방법.

비지역 변수에 접근하는 방법

Step 1. 스택에 어떤 ARI에 있는지 찾는다

Step 2. ARI 안에 변수 중 몇 번째에 있는지 찾는다(local_offset) 사용.

Static Chain과 Display 두 가지 방법이 있다.

Static Chain 정적 체임

Dynamic link처럼 static link가 내 구조의 상위인 것을 가르침.

컴파일시간에 변수가 지역인지 비지역인 것 뿐 아니라, static chain의 길이를 알 수 있다.

Static_Depth

지금 SP(내가)이 몇 번째 깊이에 있는가?

Nesting_Depth == chain offset

나로부터 참조하려는 것은 얼마나 멀리(몇 개 깊이) 떨어져 있는가?

- Actual reference : (chain_offset, local_offset)

```

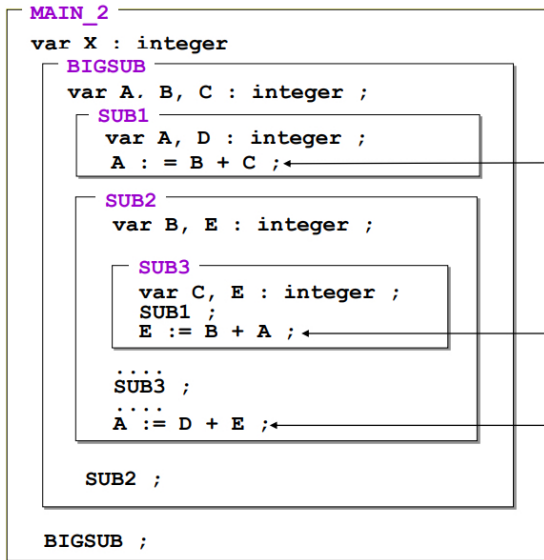
program A ;
  procedure B ;
    procedure C ;
    end; { of procedure C }
  end; { of procedure B }
end ;

```

- static_depth
A : 0, B : 1, C : 2

-chain_offset when C refers
the variable in A : 2

실제 참조 시에는 (chain_offset; 몇 번 위로 올라가야 하는지, local_offset; 그 안에 어디있는지-



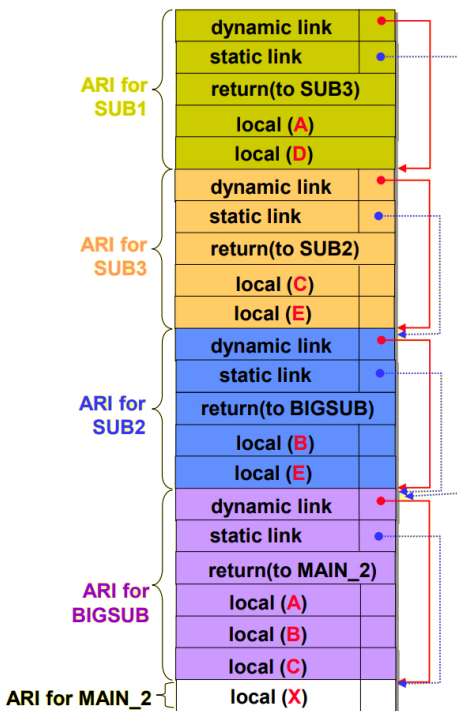
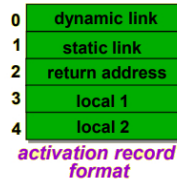
- Calling Sequence
 MAIN_2 calls BIGSUB
 BIGSUB calls SUB2
 SUB2 calls SUB3
 SUB3 calls SUB1

A : (0, 3)
 B : (1, 4)
 C : (1, 5)

E : (0, 4)
 B : (1, 3)
 A : (2, 3)

A : (1, 3)
 D : ? (Error)
 E : (0, 4)

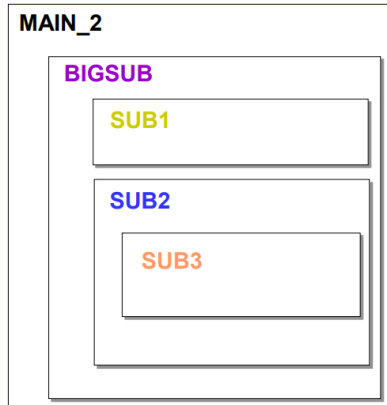
(chain offset, local-offset)



Calling Sequence

MAIN_2 -> BIGSUB -> SUB2 ->
 SUB3 -> SUB1

Program Structure



컴퓨터는 어떻게 Static Chain을 관리할까? 처음 인스턴스가 만들어졌을 때 그 인스턴스의 static link를 어떻게 찾지?

방법 1. dynamic chain을 따라가 본다. 구조적 상위에 있는게 있을때까지 찾아본다. -> 오버헤드

방법 2. 컴파일 시간에 Static depth를 다 계산을 해놓음. 그걸 보고 비지역 변수로 찾아가.

Static Chain의 단점

비지역 변수를 찾으러 탐색하는데 시간이 많이 걸림. 깊이가 깊으면 더 오래 걸림.

코드를 변경하기 전과 변경한 후 얼마나 걸릴지 예측할 수 없음.

Display 디스플레이

:static link을 ARI에 놓지 않고 별도의 array에 보관하는 방법

(display_offset, local_offset)

Display array는 static_depth별로 ARI를 가리키고 있다.

같은 Depth가 들어오면 나중에 들어온 곳을 가리킴

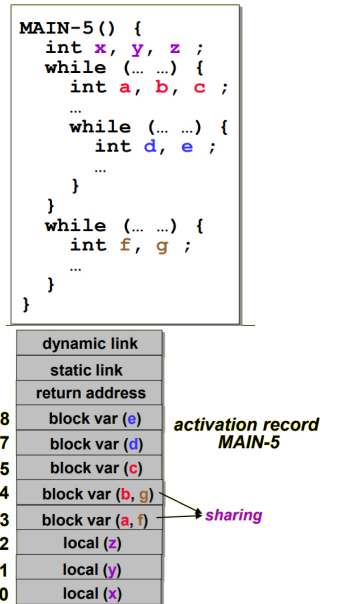
부를 ARI가 더 깊은 곳에 있으면? -> Display가 새롭게 추가되고

Display 배열의 크기는 컴파일 시간에 정해진다. 컴퓨터가 최대 static_depth를 계산함.
 Display 배열의 값은 실행시간에 계속 변함.

Static chain과 같이 간접적인 참조. but Display는 깊이에 상관없이 일정한 처리속도를 가진다. -> 빠르다
 속도가 빠르기 때문에 display를 레지스터에 둬.

1계층 차이냐면 static chain이나 display나 속도가 동일할 것.
 2계층 이상 차이냐면 display가 더 빠르다.

중첩이 깊지 않다면 static chain이 낫다. 일반적인 상황이라면.
 깊은 정적 nesting이 있고, 먼 거리에 참조를 해야 한다면 display가 낫다.



Block 블록

block = 복합문 + 데이터 선언

방법1. 매개변수 없는 procedure처럼 취급. ARI를 포함한 스택에 넣기
 스택에 값이 들어가야 해서 스택의 크기가 커짐. 리소스를 많이 사용하게 됨.

방법2. 이미 있는 ARI안에 블록을 포함시키는 방법.

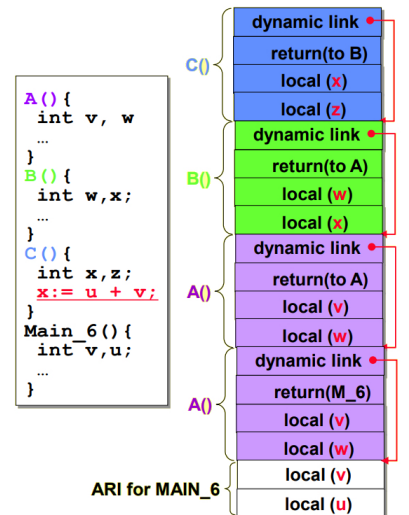
지역변수 전에 block이라고 쓰고 데이터를 넣음. 같은 시간에 쓸 수 없는 것들은 공유! ----^

Dynamic Scoping 동적 스코핑의 구현

1. Deep Access 딥 액세스

dynamic chain을 따라 감.
 chain을 따라 가서 원하는 변수가 있는지 탐색.

구현 자체는 쉽지만, 컴파일 시간에 시간이 얼마나 걸릴지 예측할 수 없음.
 일반적으로 Static scope보다 느림.

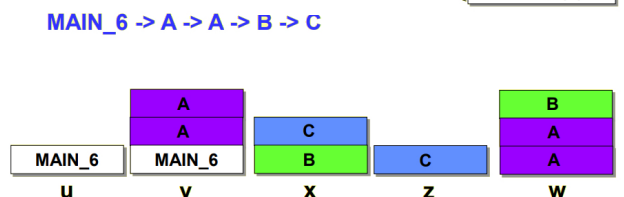
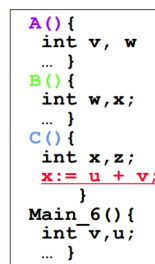


2. Shallow Access 얕은 액세스

변수를 활성화레코드에 저장하지 않음.
 각 변수들마다 스택을 따로 만들.

빠른 참조가능!

스택에서 맨 위에 있는 것(나중에 쌓인 것이)만 찾으면 그것이 참고하고자 하는 값이다!



하지만 변수별로 각각 stack을 만들어야 하므로 시공간적으로 expensive.
 기능 구현(push pop...)도 해야 하고 런타임에 오버헤드

11장 - Abstract Data Types and Encapsulation Constructs

추상 데이터타입

프로그래밍이 어려운 이유는 데이터를 그대로 표현하기 힘들기 때문.

추상화 Abstraction의 개념

프로세스 또는 객체의 일부 범주는 해당 속성의 하위 집합으로 표현될 수 있음.

프로그램에 복잡성에 대응하는 무기

필수 속성에 집중하고 하위 속성을 무시할 수 있음.

추상화의 개념은 상대적으로 간단하지만, 추상화를 완벽히 지원하게 된 것은 비교적 최근이다.

->추상 데이터 타입으로 구현

추상화의 두 가지 종류

프로세스 추상화

함수

어떤 프로세스가 어떻게 동작하는지 감추는 것

프로세스의 내부가 어떻게 만들어지고 어떻게 구현됐는지 알 필요 없음.

sort_int(list,length) : 여기서 버블 소트로 만들어졌는지 퀵소트로 만들어졌는지 알 방법 없음.

데이터 추상화

클래스

표현과 구현에 대한 세부사항을 프로그래머로부터 숨기는 것

데이터 추상화

데이터 추상화는 프로세스 추상화보다 더 늦게 나온 개념임.

->data-oriented programming

추상 데이터 타입으로서 Floating-Point

모든 built-in 기본타입된 타입은 추상화 데이터 타입이다. - float도

선언하는 방법, 안에서 지원하는 연산(자)은 프로그래머가 알아야 함.

Information Hiding 정보은폐 : 몰라도 되는 정보를 사용자가 볼 수 없게 감추는 것. float의 구현 방식 등..

+ 사용자가 직접적으로 실제 표현(구현)을 조작, 수정하는 것을 허용하지 않음.

시스템에서 제공하는 연산만 할 수 있음. 데이터 타입에 대해서 사용자가 새로운 연산자 만드는 것 불허

데이터 표현을 유연(flexible)하게 할 수 있다.

서로 다른 기계에서 다른 방식으로 내부구현이 되어 있더라도 선언과 사칙연산은 같기 때문에 호환.

portability(이식성)있음.

User-defined 사용자 구현 추상화 데이터 타입

추상데이터 타입의 조건 2가지

1. 표현, 정의, 연산이 한 구문 단위(한 덩어리)로 묶여야 함. (Encapsulation 캡슐화)

<-> 한 그룹, 한 컴파일 단위

2. 추상화한 데이터타입을 '사용하는 프로그램 단위'로부터 숨겨져야 함. (Information Hiding)

e.g. 객체가 클래스의 데이터에 직접 접근하면 안된다. 함수 등으로 간접적으로 접근해야 함.

추상화를 함으로서 얻을 수 있는 장점 2가지

1. Localized modification 지역적 수정 (by Encapsulation)

e.g. 클래스 내부에서 선언된 것은 내부에서 밖에 수정하지 못함. 외부에서 수정 불가능.

2. 신뢰성(Reliability) 향상 (by Information Hiding)

프로그램 유닛이 표현을 직접적으로 내부를 변경할 수 없고, 메소드(접근함수) 등을 써야만 가능

무결성(Integrity) 향상. 고의적, 실수로 값을 건드릴 수 없음.

stack 스택; 추상 데이터 타입 예

⇔ `create(stack)`
⇔ `destroy(stack)`
⇔ `empty(stack)`
⇔ `push(stack, element)`
⇔ `pop(stack)`
⇔ `top(stack)`

데이터 표현 방식이 아니라, 추상적인 속성에만 의존.
stack의 구현을 array에서 linked list로 바뀌도 상관X

추상 데이터 타입의 디자인 이슈

타입을 정의해야 하고, 추상화 연산의 서브프로그램을 하나의 구문 단위로 묶을 수 있어야 한다.(encapsulation)
외부(클래스 외부)에서 어디까지 볼 수 있는지 정의해야 함.
할당(assignment)나 비교연산을 어떻게 해야 할지 구현을 해야 함.

언어에 따라서 구현 방법이 2가지로 나뉨

1. 단일 데이터타입과 연산을 사용자가 직접 할 수 있게 함. 선택해서 가져올 수 있는 방법?
Pascal, Smalltalk, C++
2. 객체를 정의할 수 있는 일반적인 방법 제공.
Modula-2, Ada

디자인 이슈

1. 추상적일 수 있는 타입을 어떻게 제한할 것인가
2. **Generic**하게 쓸 수 있는가? 다양한 데이터타입에 대해 쓸수 있는가?
3. 지역이름과 비지역이름이 충돌할 때 어떻게 대응할 것인가?

SIMULA 67

데이터 추상화를 직접적으로 지원한 첫 번째 언어

"class"

단점 : 클래스에 대한 인스턴스가 동적으로만 할당될 수 있었다. -> 포인터로만 접근할 수 있었음.
클래스 인스턴스의 "code section"이 시작할 때 한번만 실행이 된다. (생성자)

캡슐화 : 데이터 선언과 procedures들을 구문적으로 캡슐화 함.

정보은폐 : 클래스 내부에서 선언된 변수들이 외부로부터 감춰지지 않았음! 외부에서 접근 가능했음. -> 문제!

Modula-2

정보은폐 기능이 추가

"module"

캡슐화:

module은 타입에 대한 정의, 객체, SP까지 포함되게 만듦.

definition module(정의모듈) : 정의하는 것만 모여있는 모듈 ->외부에서 봐도 상관X

implementation module(구현모듈) : 구현되는 내용만 모여있는 모듈 ->외부에서 보면 안 됨! 감추기

>>C에서 정의 따로 구현 따로와 비슷

모듈들은 별도로 컴파일 가능했음.

정보은폐 :

transparent 투명 : 정의모듈에 구현하는 내용까지 모두 넣어놓은 방식

opaque 불투명 : 엄격하게 정의는 정의모듈에, 구현은 구현모듈에 해놓은 방식 - 포인터형태로만 구현가능!

-> 추상 데이터 타입은 opaque 불투명 해야 함.

모듈에서 일부분만 불러와서 사용이 가능함.

이름이 충돌하면 명시적으로 `stackmod.push()`, 아니면 묵시적으로 `push()` 사용 가능

Ada => 최초의 완벽한 추상 데이터 타입 지원

"package"

캡슐화 :

specification package 명세 패키지 - 정의 모듈과 비슷

body package 바디 패키지 - 구현 모듈과 비슷

정보은폐 :

private 등 접근지정자 사용함. 이를 통해 정보은폐를 함. 단점은 private는 무조건 패키지 마지막에 써야 함.

Modula-2와의 차이

Modula-2는 불투명하게 하려면 반드시 포인터를 사용했어야... 하지만 Ada는 제한 없음.

Modula-2는 특정 entities(일부)만 가져올 수 있었지만, Ada는 지원하지 않음.

C++

"class"

캡슐화 :

data member 데이터 멤버

member function(method) 멤버 함수

모든 인스턴스는 각자끼리만 변수를 공유함.

클래스 인스턴스(오브젝트)는 static, stack dynamic, heap dynamic 3가지로 나뉨.

static 함수 바깥에서 정적으로 선언된 인스턴스

stack dynamic 함수 안에서 선언된 인스턴스

heap dynamic new, delete 등으로 동적으로 선언된 인스턴스

정보은폐:

private : 숨김 public : 볼수 있게 함.(인터페이스)

protected : 숨기지만, 상속받은 것은 볼수 있음

constructor 함수 : 인스턴스를 생성하는 순간 자동으로(묵시적) 실행되는 함수

stack()

destructor 함수 : 인스턴스의 생명이 끝날 때 자동적으로(묵시적) 실행되는 함수

~stack()

명시적으로 생성자 소멸자 호출할 수도 있다.

Java

C++의 차이점.

자바는 struct가 없어서 사용자 정의 데이터 타입을 쓰기 위해서는 무조건 class를 사용해야 함.

자바는 모든 객체가 **heap dynamic**으로 고정되어 있음.

클래스 내부에 선언과 구현까지 같이 해야 함. <-> C++은 원한다면 따로 할 수 있음.

자바는 소멸자가 없음. 쓰레기 수집기(가비지 콜렉팅)이 알아서 해줌.

매개변수화된 추상 데이터 타입

generic : 한번 정의한 데이터 구조를 다양한 데이터 타입에 재사용할 수 있음

generic하게 만든 추상 데이터 타입.

정적 타입 언어에서만 구현 필요. 파이썬같이 동적 타입인 언어에서는 필요X